Week 4: Implement Hill Climbing search algo to solve N-Queens problem.

Algorithm:

function HILL-CLIMBING (problem) return a state which is a local maximum

current ← make-node (problem, INITIAL-STATE)

loop do

neighbor ← a highest valued successor of current

if neighbor.VALUE ≤ current.VALUE

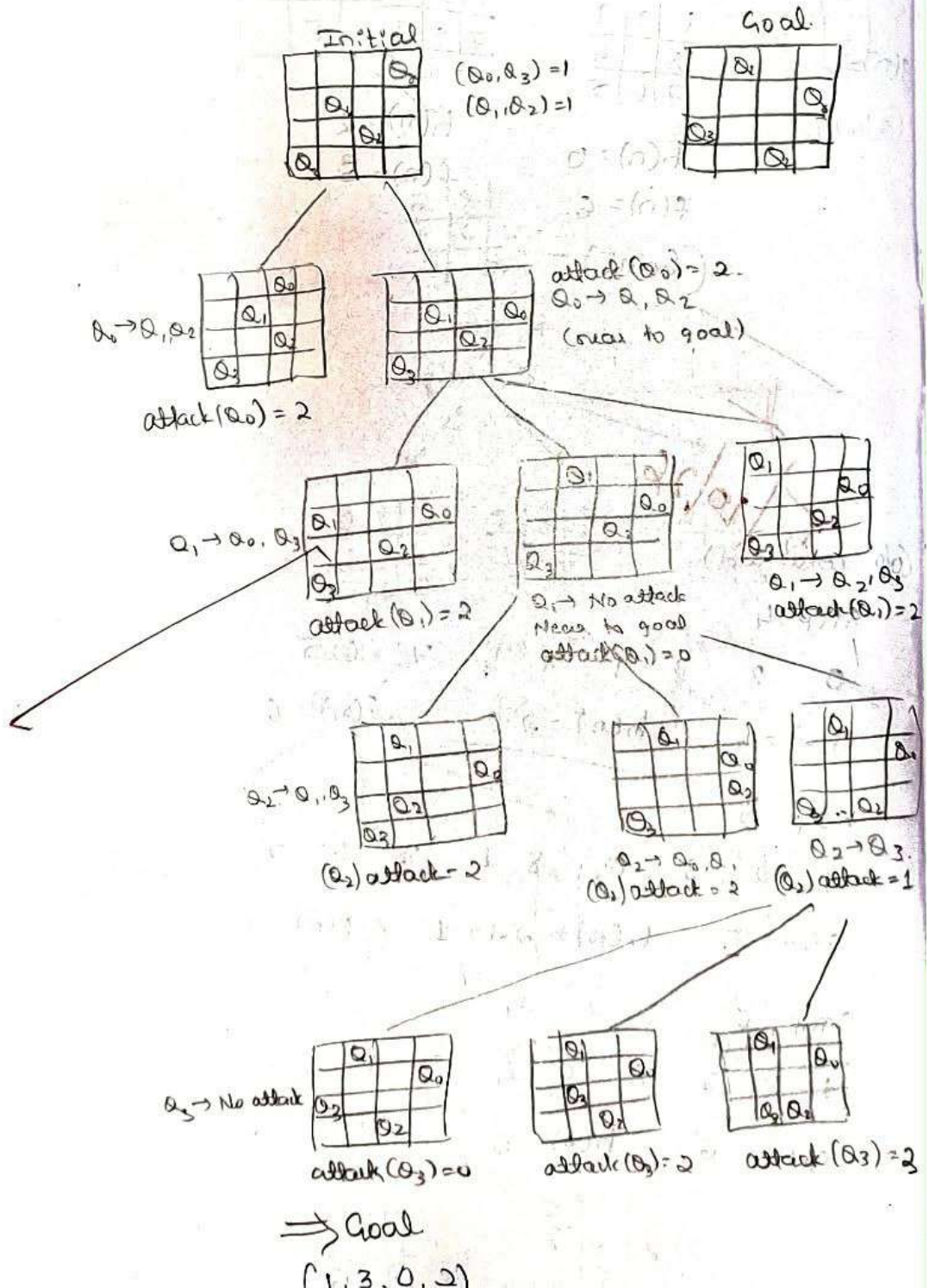
then return current.STATE

current = neighbor.

Solution:

$x_0 = 3$ $x_2 = 2$

$x_1 = 1$ $x_3 = 0$



⇒ Goal

(1, 3, 0, 2)

Output :

$(x_0, x_1, x_2, x_3) = (3, 1, 2, 0)$ cost = 0.

1) Swapped Queen	Neighbor state	cost
x_0 x_1	(1, 3, 2, 0)	1
x_0 x_2	(2, 1, 3, 0)	1
x_0 x_3	(0, 1, 2, 3)	6
x_1 x_2	(3, 2, 1, 0)	6
x_1 x_3	(3, 0, 2, 1)	1
x_2 x_3	(3, 1, 0, 2)	1

min = 1 (x_0, x_1) → min. neighbor subscript

2) x_1 & x_2 can attack each other.

x_0 x_1	(3, 1, 2, 0)	2
x_0 x_2	(2, 3, 1, 0)	2
x_0 x_3	(0, 3, 2, 1)	4
x_1 x_2	(1, 2, 3, 0)	4
x_1 x_3	(1, 0, 2, 3)	2
x_2 x_3	(1, 3, 0, 2)	0 → min = 0. No attack Goal stat

3) current ~~state~~ : [1, 3, 0, 2]

~~Goal~~ current cost = 0.

Goal state reached!

Final state : [1, 3, 0, 2]

Final cost : 0.

```
Enter number of queens: 4
Enter row positions for each queen (0 to 3 )
Enter row for queen x0: 3
Enter row for queen x1: 1
Enter row for queen x2: 2
Enter row for queen x3: 0

Initial state: [3, 1, 2, 0]
Initial cost: 2

Step 1
Current state: [3, 1, 2, 0]
Current cost: 2

Neighbor states:
Swap x0, x1 -> [1, 3, 2, 0] Cost: 1
Swap x0, x2 -> [2, 1, 3, 0] Cost: 1
Swap x0, x3 -> [0, 1, 2, 3] Cost: 6
Swap x1, x2 -> [3, 2, 1, 0] Cost: 6
Swap x1, x3 -> [3, 0, 2, 1] Cost: 1
Swap x2, x3 -> [3, 1, 0, 2] Cost: 1

Step 2
Current state: [1, 3, 2, 0]
Current cost: 1

Neighbor states:
Swap x0, x1 -> [3, 1, 2, 0] Cost: 2
Swap x0, x2 -> [2, 3, 1, 0] Cost: 2
Swap x0, x3 -> [0, 3, 2, 1] Cost: 4
Swap x1, x2 -> [1, 2, 3, 0] Cost: 4
Swap x1, x3 -> [1, 0, 2, 3] Cost: 2
Swap x2, x3 -> [1, 3, 0, 2] Cost: 0

Step 3
Current state: [1, 3, 0, 2]
Current cost: 0
Goal state reached!

Final state: [1, 3, 0, 2]
Final cost: 0
Deeksha Raju Naik(1WN24CS086)
```

Week - 5

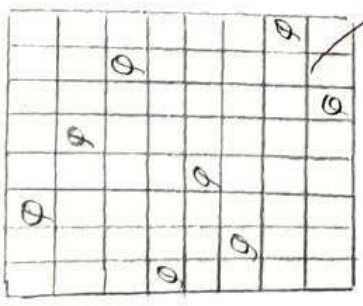
Simulated Annealing using 8 Queens.

```
current ← initial state
T ← a large positive value
while T > 0 do
        next ← a random neighbor of current
        ΔE ← current cost - next cost
        if ΔE < 0 then
                current ← next
        else
                current ← next with probability p e^(ΔE/t)
        end if
        decrease T
end while
return element
```

Enter the row position of each queen (1-8):

Queen 1 : 3
Queen 2 : 1
Queen 3 : 2
Queen 4 : 0
Queen 5 : 7
Queen 6 : 5
Queen 7 : 6
Queen 8 : 4

Final Board:



Queen placements : [6, 4, 2, 8, 5, 7, 1, 3]

Conflicts = 0

Solution found!

```
Enter the row position of each queen (1-8):
Queen 1: 3
Queen 2: 1
Queen 3: 2
Queen 4: 0
Queen 5: 7
Queen 6: 5
Queen 7: 6
Queen 8: 4

Final Board:
. . . . . . Q .
. . Q . . . . .
. . . . . . . Q
. Q . . . . . .
. . . . Q . . .
Q . . . . . . .
. . . . . Q . .
. . . Q . . . .

Queen placements: [6, 4, 2, 8, 5, 7, 1, 3]
Conflicts: 0
Solution found!
DEEKSHA R NAIK 1WN24CS086
```

Week -6.

Create a knowledge base using propositional logic and show that the given query entails the knowledge base or not.

Algorithm:

function TT-ENTILES? (KB, α) returns true or false

inputs KB, the knowledge base, a sentence in Proposition α, the query, a sentence in proposition logic.

Symbols ← a list of propositon symbols in KB & α

return TT_CHECKALL (KB, α, symbols, {})

function TT_CHECK_ALL (KB, α, symbols, models) returns true or false

if empty? (symbols) then

if PL-TRUE? (KB, model) then returns PL-TRVE? (α, model)

else returns TRUE

else do

P ← FIRST

rest ← REST

return (TT_CHECK-ALL (KB, α, rest, model ∪ {P = true})

and TT_CHECK-ALL (KB, α, rest, model ∪ {P = false})

Consider a knowledge base KB that contains the following Propositional logic sentences:

$Q \rightarrow P$

$P \rightarrow \neg Q$

$Q \vee R$

i) Truth Table: $KB = (Q \rightarrow P) \wedge (P \rightarrow \neg Q) \wedge (Q \vee R)$

P	Q	R	$\neg Q$	$Q \rightarrow P$	$P \rightarrow \neg Q$	$Q \vee R$	KB	$R \rightarrow P$	Q
T	T	T	F	T	F	T	F	T	
T	T	F	F	T	F	T	F	T	
T	F	T	T	T	T	T	T	T	T
T	F	F	T	T	T	F	F	T	T
F	T	T	F	F	T	T	F	F	
F	T	F	F	F	T	T	F	T	F
F	F	T	T	T	T	T	T	F	T
F	F	F	T	T	T	F	F	T	T

ii) Does KB entail R?

$KB \models R \quad (\alpha = R)$ i.e. $KB \models \alpha$.

Condition satisfied

∴ KB entails R

iii) KB entails $R \rightarrow P$?

$KB \not\models R \rightarrow P$: False

$R \rightarrow P$ is not True when KB = True.

∴ KB does not entail $R \rightarrow P$

iv) KB entails $Q \rightarrow R$?

$KB \models Q \rightarrow R$

Holds true.

$Q \rightarrow R$ is true when KB is true.

∴ KB entails $Q \rightarrow R$.

02/11/26

```
P       Q       R       Q->P    P->~Q   Q|R     KB
False   False   False   True    True    False   False
False   False   True    True    True    True    True
False   True    False   False   True    True    False
False   True    True    False   True    True    False
True    False   False   True    True    False   False
True    False   True    True    True    True    True
True    True    False   True    False   True    False
True    True    True    True    False   True    False

Models of KB:
(False, False, True)
(True, False, True)

KB entails R: True
KB entails R -> P: False
KB entails Q -> R: True
DEEKSHA R NAIK (1WN24CS086)
```